

江西财经大学

2025-2026 学年第 3 学期

《C++ 综合实践》实训习题

课程代码 1005405122 选 课 班 软工 VR255 班

课程名称 C++ 综合实践 任课教师 张勇

学 号 0256658 姓 名 桑赫

学 院 虚拟现实 (VR) 现代产业学院

专 业 软件工程 (VR 软件开发)

2026 年 6 月 14 日

实训题 1

设有 4 个学生，每个学生有 5 门课的成绩，要求从键盘输入学生的学号、姓名及 5 门课的成绩，计算出平均成绩，并将每个学生的全部数据以字符串的形式保存到磁盘上的文本文件 StudentGrade.txt 中。

```
1
2 #include<iostream>
3 using namespace std;
4 #include<string>
5 #include<iomanip>
6 #include<cstdlib>
7 #include<fstream>
8 #include<sstream>
9
10 class Student{
11 private:
12     string m_name;
13     int m_id;
14     int grades[5];
15     double m_average;
16
17 public:
18
19     void input(){
20         cout<<"请输入学生姓名: "<<endl;
21         cin>>m_name;
22         this->m_name;
23         cout<<"请输入学生学号: "<<endl;
24         cin>>m_id;
25         this->m_id;
26
27         int sum=0;
28         for(int i=0;i<5;i++){
29             cout<<"请输入第"<<i+1<<"门课程成绩: "<<endl;
30             cin>>this->grades[i];
31             sum+=this->grades[i];
32         }
33         this->m_average=(double)sum/5;}
34
35     string toString() const {
36         ostringstream oss;
```

```
37         oss << m_name << "; " << m_id;
38         for(int i = 0; i < 5; i++) {
39             oss << "; " << grades[i];
40         }
41         oss << "; " << fixed << setprecision(2) << m_average;
42         return oss.str();
43     }
44
45     void write(){
46         ofstream ofs("StudentGrade.txt", ios::app);
47         if(!ofs) {
48             cerr << "无法打开文件 StudentGrade.txt" << endl;
49             return;
50         }
51         ofs << toString() << endl;
52
53         ofs.close();
54     }
55
56
57 };
58 int main(){
59     system("chcp 65001 > nul");
60
61     for(int i = 0; i < 4; i++){
62         cout << "输入第" << i+1 << "位学生信息" << endl;
63         Student s;
64         s.input();
65         s.write();
66     }
67
68     return 0;
69 }
```

```
--- 输入第 1 位学生信息 ---  
请输入学生姓名：  
张三  
请输入学生学号：  
34  
请输入第1门课程成绩：  
67  
请输入第2门课程成绩：  
57  
请输入第3门课程成绩：  
79  
请输入第4门课程成绩：  
67  
请输入第5门课程成绩：  
89  
该学生信息已保存
```

图 1: 实验一运行结果

实训题 2

读取上题中的文本文件 StudentGrade.txt 中的数据，按平均成绩进行降序排序，并将排序后的所有数据保存到新的文本文件 SortedGrade.txt 中。

提示：所有学生的数据保存到字符串数组中，字符串数组的每个元素是一个字符串，包含了一个学生的所有数据，平均成绩应该是出现在第 7 个；号到本行结束之间的所有字符，读出平均成绩，按成绩对字符串数组排序。

```
1 #include<iostream>  
2 #include<string>  
3 #include<iomanip>  
4 #include<cstdlib>  
5 #include<fstream>  
6 #include<vector>  
7 #include<algorithm>  
8  
9 using namespace std;  
10  
11 static double extractAverage(const string &line) {  
12     const char delimiter = ';';
```

```
13     int count = 0;
14     size_t pos = string::npos;
15     for(size_t i = 0; i < line.size(); i++) {
16         if(line[i] == delimiter) {
17             count++;
18             if(count == 7) {
19                 pos = i;
20                 break;
21             }
22         }
23     }
24     if(pos == string::npos || pos + 1 >= line.size()) {
25         return 0.0;
26     }
27     string avgStr = line.substr(pos + 1);
28     try {
29         return stod(avgStr);
30     } catch(...) {
31         return 0.0;
32     }
33 }
34
35 int main() {
36     system("chcp 65001 > nul");
37
38     vector<string> records;
39     ifstream ifs("StudentGrade.txt");
40     if(!ifs) {
41         cerr << "无法打开文件 StudentGrade.txt" << endl;
42         return 1;
43     }
44
45     string line;
46     while(getline(ifs, line)) {
47         if(!line.empty()) {
48             records.push_back(line);
49         }
50     }
51     ifs.close();
52
53     sort(records.begin(), records.end(), [](const string &a, const string &b) {
```

```
54         return extractAverage(a) > extractAverage(b);
55     });
56
57     ofstream ofs("SortedGrade.txt", ios::out);
58     if(!ofs) {
59         cerr << "无法创建文件 SortedGrade.txt" << endl;
60         return 1;
61     }
62
63     for(const auto &record : records) {
64         ofs << record << endl;
65     }
66     ofs.close();
67
68     cout << "排序完成，结果已保存到 SortedGrade.txt" << endl;
69     return 0;
70 }
```

```
张三; 1; 67; 87; 56; 78; 76; 72.80
王五; 4; 65; 84; 78; 99; 15; 68.20
李四; 5; 91; 45; 65; 28; 15; 48.80
徐盛; 98; 67; 79; 65; 25; 45; 56.20
```

图 2: 实验二运行结果

实训题 3

构造结构体 StudentGrade，要求从键盘输入学生的学号、姓名及 5 门课的成绩，计算出平均成绩，写入数据到结构体数组中。并将每个学生的全部数据保存到磁盘上的二进制文件 StudentGrade.dat 中。

提示：二进制文件创建语句 pf=fopen("student.dat","wb");

写二进制文件语句 fwrite(&stu[i],sizeof(Student),1,fp);

```
1  #include <iostream>
2  #include <cstdio>
3  #include <cstring>
4  using namespace std;
5
6  struct StudentGrade {
7      int id;
8      char name[64];
9      int scores[5];
10     double average;
11 };
12
13 int main() {
14     system("chcp 65001 > nul");
15
16     int n;
17     cout << "请输入学生数量：";
18     cin >> n;
19     if(n <= 0) {
20         cerr << "学生数量必须大于0。" << endl;
21         return 1;
22     }
23     if(n > 100) {
24         cerr << "最大支持100名学生。" << endl;
25         return 1;
26     }
27
28     StudentGrade *students = new StudentGrade[n];
29
30     for(int i = 0; i < n; i++) {
31         cout << "\n请输入第" << i + 1 << "名学生的学号：";
32         cin >> students[i].id;
33         cout << "请输入第" << i + 1 << "名学生的姓名：";
34         cin >> students[i].name;
```

```
35     int sum = 0;
36     for(int j = 0; j < 5; j++) {
37         cout << "请输入第" << i + 1 << "名学生第"
38             << j + 1 << "门课程成绩：";
39         cin >> students[i].scores[j];
40         sum += students[i].scores[j];
41     }
42     students[i].average = static_cast<double>(sum) / 5.0;
43 }
44
45 FILE *fp = fopen("StudentGrade.dat", "wb");
46 if(!fp) {
47     cerr << "无法创建二进制文件 StudentGrade.dat" << endl;
48     delete[] students;
49     return 1;
50 }
51
52 for(int i = 0; i < n; i++) {
53     fwrite(&students[i], sizeof(StudentGrade), 1, fp);
54 }
55
56 fclose(fp);
57 delete[] students;
58 cout << "已将 " << n << " 名学生的全部数据保存到 StudentGrade.dat"
59     << endl;
60 return 0;
61 }
```



```
请输入第1名学生的学号: 79
请输入第1名学生的姓名: vstp
请输入第1名学生第1门成绩: 45
请输入第1名学生第2门成绩: 43
请输入第1名学生第3门成绩: 78
请输入第1名学生第4门成绩: 89
请输入第1名学生第5门成绩: 78

请输入第2名学生的学号: 09
请输入第2名学生的姓名: pia
请输入第2名学生第1门成绩: 65
请输入第2名学生第2门成绩: 32
请输入第2名学生第3门成绩: 95
请输入第2名学生第4门成绩: 45
请输入第2名学生第5门成绩: 62

请输入第3名学生的学号: 16
请输入第3名学生的姓名: lec
请输入第3名学生第1门成绩: 87
请输入第3名学生第2门成绩: 23
请输入第3名学生第3门成绩: 56
请输入第3名学生第4门成绩: 42
请输入第3名学生第5门成绩: 93
已将 3 名学生的全部数据保存到 StudentGrade.dat
```

图 3: 实验三运行结果

实训题 4

设计一个函数 SortByAvg(), 读取上题中的二进制文件 StudentGrade.dat 中的数据到结构体数组中, 按平均成绩进行降序排序, 并将排序后的所有数据保存到新的二进制文件 SortedGrade.dat 中。

```
1 #include<iostream>
2 #include<cstdio>
3 #include<cstring>
4 #include<algorithm>
5 using namespace std;
6
7 struct StudentGrade {
8     int id;
9     char name[64];
10    int scores[5];
11    double average;
```

```
12 };
13
14 void SortByAvg(const char *inFile = "StudentGrade.dat",
15 const char *outFile = "SortedGrade.dat"){
16     FILE *fp = fopen(inFile, "rb");
17     if(!fp){
18         cerr << "无法打开输入文件 " << inFile << endl;
19         return;
20     }
21     if(fseek(fp, 0, SEEK_END) != 0){ fclose(fp);
22     cerr << "读取文件长度失败 " << endl; return; }
23     long sz = ftell(fp);
24     if(sz <= 0){ fclose(fp);
25     cerr << "输入文件为空或无法读取大小 " << endl; return; }
26     size_t count = static_cast<size_t>(sz / sizeof(StudentGrade));
27     rewind(fp);
28
29     StudentGrade *arr = new StudentGrade[count];
30     size_t read = fread(arr, sizeof(StudentGrade), count, fp);
31     fclose(fp);
32     if(read == 0){ cerr << "未读取到任何记录 " << endl;
33     delete[] arr; return; }
34
35     sort(arr, arr + count, []
36     (const StudentGrade &a, const StudentGrade &b)
37     { return a.average > b.average; });
38
39     FILE *ofp = fopen(outFile, "wb");
40     if(!ofp){ cerr << "无法创建输出文件 " << outFile << endl;
41     delete[] arr; return; }
42     fwrite(arr, sizeof(StudentGrade), count, ofp);
43     fclose(ofp);
44     delete[] arr;
45     cout << "已将排序后的数据保存到 " << outFile << endl;
46 }
47
48 int main(){
49     system("chcp 65001 > nul");
50     SortByAvg();
51     return 0;
52 }
```

===== 按平均成绩降序排序结果 =====							
姓名	学号	成绩1	成绩2	成绩3	成绩4	成绩5	平均分
李四	1002	95	88	94	90	89	91.20
张三	1001	85	78	92	80	77	82.40
赵六	1004	80	72	78	74	75	75.80
王五	1003	70	65	72	68	68	68.60
=====							
共 4 名学生							

图 4: 实验四运行结果

实训题 5

编写一个随机加分的程序，要求如下：

(1) 事先编辑好一个存放所有学生学号的文本文件，每个学生的学号占一行，假设总行数为 m ；

(2) 产生一个随机数 $n(1 \leq n \leq m)$ ，据此读取文本文件中的第 n 行的学生学号并输出，然后删除这个文本文件的第 n 行，设置总行数为 $m-1$ ；

(3) 在学生结构体数组中查询刚才随机挑选的学号的学生记录是否存在，如果存在则将该生的数学成绩加 10 分，并更新磁盘上的数据文件，如果不存在则提示查无此人。

(4) 将该功能实现为循环模式，可用户决定是否继续随机加分。

解答：

```
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <string>
5 #include <cstdlib>
6 #include <ctime>
7 #include <cstdio>
8 #include <cstring>
9 using namespace std;
10
11 struct StudentGrade {
12     int id;
13     char name[64];
14     int scores[5];
15     double average;
16 };
17
18 static bool readIdLines(const string &file, vector<string> &lines){
19     lines.clear();
20     ifstream ifs(file);
21     if(!ifs) return false;
```

```
22     string ln;
23     while(getline(ifs, ln)){
24         if(!ln.empty()) lines.push_back(ln);
25     }
26     return true;
27 }
28
29 static bool writeIdLines(const string &file,
30     const vector<string> &lines){
31     ofstream ofs(file, ios::out | ios::trunc);
32     if(!ofs) return false;
33     for(size_t i=0;i<lines.size();++i) ofs << lines[i] << "\n";
34     return true;
35 }
36
37 static bool loadStudents(const char *file, vector<StudentGrade> &arr){
38     arr.clear();
39     FILE *fp = fopen(file, "rb");
40     if(!fp) return false;
41     if(fseek(fp, 0, SEEK_END) != 0){ fclose(fp); return false; }
42     long sz = ftell(fp);
43     if(sz <= 0){ fclose(fp); return false; }
44     size_t count = static_cast<size_t>(sz / sizeof(StudentGrade));
45     rewind(fp);
46     arr.resize(count);
47     size_t read = fread(arr.data(), sizeof(StudentGrade), count, fp);
48     fclose(fp);
49     return read == count;
50 }
51
52 static bool saveStudents(const char *file,
53     const vector<StudentGrade> &arr){
54     FILE *fp = fopen(file, "wb");
55     if(!fp) return false;
56     size_t written = fwrite(arr.data(),
57         sizeof(StudentGrade), arr.size(), fp);
58     fclose(fp);
59     return written == arr.size();
60 }
61
62 int main(){
```

```
63     system("chcp 65001 > nul");
64     const string idFile = "StudentIDs.txt"; // 每行一个学号
65     const char *binFile = "StudentGrade.dat";
66
67     srand((unsigned)time(NULL));
68
69     while(true){
70         vector<string> ids;
71         if(!readIdLines(idFile, ids)){
72             cerr << "无法打开或读取文件 " << idFile << endl;
73             return 1;
74         }
75         size_t m = ids.size();
76         if(m == 0){
77             cout << "学号文件为空，退出。" << endl;
78             break;
79         }
80         size_t n = (rand() % m); // 0-based index
81         string chosen = ids[n];
82         cout << "随机抽取到的学号：" << chosen << endl;
83
84         ids.erase(ids.begin() + n);
85         if(!writeIdLines(idFile, ids)){
86             cerr << "删除行后无法写回学号文件" << endl;
87             return 1;
88         }
89
90         vector<StudentGrade> students;
91         if(!loadStudents(binFile, students)){
92             cerr << "无法读取二进制学生文件或文件不存在：" << binFile
93                 << endl;
94
95             break;
96         }
97         int targetId = 0;
98         try{
99             targetId = stoi(chosen);
100         }catch(...){
101             cerr << "学号不是有效整数：" << chosen << endl;
102
103             cout << "是否继续随机加分？(y/n)：";
```

```
104         string ans; cin >> ans; if(ans!="y" && ans!="Y") break;
105             else continue;
106     }
107
108     bool found = false;
109     for(size_t i=0;i<students.size();++i){
110         if(students[i].id == targetId){
111             cout << "找到学号 " << targetId
112                 << ", 原数学成绩: "
113                 << students[i].scores[0] << endl;
114             students[i].scores[0] += 10;
115             cout << "加分后数学成绩: "
116                 << students[i].scores[0] << endl;
117
118             int sum = 0;
119             for(int k=0;k<5;++k) sum += students[i].scores[k];
120             students[i].average = static_cast<double>(sum) / 5.0;
121             found = true;
122             break;
123         }
124     }
125
126     if(found){
127         if(!saveStudents(binFile, students)){
128             cerr << "无法将更新后的学生数据写回二进制文件" << endl;
129             return 1;
130         }
131         cout << "已更新二进制文件中的成绩。" << endl;
132     }else{
133         cout << "查无此人 (学号 " << targetId << ")。" << endl;
134     }
135
136     cout << "是否继续随机加分? (y/n): ";
137     string ans; cin >> ans;
138     if(ans != "y" && ans != "Y") break;
139 }
140
141 cout << "程序结束。" << endl;
142 return 0;
143 }
```

```
随机抽取到的学号: 52
查无此人 (学号 52) 。
是否继续随机加分? (y/n): y
随机抽取到的学号: 45
查无此人 (学号 45) 。
是否继续随机加分? (y/n): y
随机抽取到的学号: 15
查无此人 (学号 15) 。
是否继续随机加分? (y/n): cd "d:\
程序结束。
```

图 5: 实验五运行结果

实训题 6

修改学生结构体，增加籍贯和专业字段，并实现以下统计功能：

- (1) 计算和报告各门课程按籍贯或按专业分组后成绩的最大值、最小值、均值、中位数。
- (2) 按成绩统计，计算和报告各门课程各成绩段人数占选课人数的百分比。成绩分为 5 段：0、(0,60)、[60,75)、[75,90)、[90,100]。

```
1 #include <iostream>
2 #include <cstdio>
3 #include <cstring>
4 #include <vector>
5 #include <string>
6 #include <map>
7 #include <array>
8 #include <algorithm>
9 #include <cmath>
10 #include <iomanip>
11 using namespace std;
12
13 struct StudentGrade {
14     int id;
15     char name[64];
16     char hometown[64];
17     char major[64];
```

```
18     int scores[5];
19     double average;
20 };
21
22 static double calculateAverage(const StudentGrade &stu) {
23     int sum = 0;
24     for(int i = 0; i < 5; i++) sum += stu.scores[i];
25     return static_cast<double>(sum) / 5.0;
26 }
27
28 static bool saveStudents(const char *file,
29 const vector<StudentGrade> &students) {
30     FILE *fp = fopen(file, "wb");
31     if(!fp) return false;
32     fwrite(students.data(),
33           sizeof(StudentGrade), students.size(), fp);
34     fclose(fp);
35     return true;
36 }
37
38 static bool loadStudents(const char *file,
39 vector<StudentGrade> &students) {
40     students.clear();
41     FILE *fp = fopen(file, "rb");
42     if(!fp) return false;
43     if(fseek(fp, 0, SEEK_END) != 0) { fclose(fp); return false; }
44     long sz = ftell(fp);
45     if(sz <= 0) { fclose(fp); return false; }
46     size_t count = static_cast<size_t>(sz / sizeof(StudentGrade));
47     rewind(fp);
48     students.resize(count);
49     size_t readCount = fread(students.data(),
50
51           sizeof(StudentGrade), count, fp);
52     fclose(fp);
53     return readCount == count;
54 }
55
56 static double medianValue(vector<int> &values) {
57     if(values.empty()) return 0.0;
58     sort(values.begin(), values.end());
```



```
59     size_t n = values.size();
60     return (n % 2 == 1)
61         ? static_cast<double>(values[n/2])
62         : (static_cast<double>(values[n/2 - 1]) +
63           static_cast<double>(values[n/2])) / 2.0;
64 }
65
66
67 static void reportGroupStatistics(const vector<StudentGrade> &students,
68 bool byHometown) {
69     const char *groupName = byHometown ? "籍贯" : "专业";
70     map<string, array<vector<int>, 5>> groupScores;
71
72     for(const auto &stu : students) {
73         string key = byHometown ? string(stu.hometown) :
74             string(stu.major);
75         for(int i = 0; i < 5; i++) {
76             groupScores[key][i].push_back(stu.scores[i]);
77         }
78     }
79
80     cout << "\n按" << groupName << "分组的课程成绩统计：" << endl;
81     for(const auto &entry : groupScores) {
82         const string &key = entry.first;
83         cout << "\n" << groupName << ": " << key << endl;
84         cout << left << setw(12) << "课程" << setw(8) << "最小"
85             << setw(8) << "最大" << setw(10) << "均值" << setw(10)
86             << "中位数" << "\n";
87         for(int c = 0; c < 5; c++) {
88             const vector<int> &values = entry.second[c];
89             if(values.empty()) continue;
90             vector<int> sorted = values;
91             sort(sorted.begin(), sorted.end());
92             int minv = sorted.front();
93             int maxv = sorted.back();
94             double sum = 0.0;
95             for(int score : sorted) sum += score;
96             double mean = sum / sorted.size();
97             double med = medianValue(sorted);
98             cout << left << setw(12) << ("Course" + to_string(c + 1))
99                 << setw(8) << minv
```

```
100         << setw(8) << maxv
101         << setw(10) << fixed << setprecision(2) << mean
102         << setw(10) << fixed << setprecision(2) << med
103         << "\n";
104     }
105 }
106 }
107
108 static void reportGradeSegments(const vector<StudentGrade> &students) {
109     const int courseCount = 5;
110     const int segmentCount = 5;
111     vector<array<int, segmentCount>> counts(courseCount);
112     for(auto &arr : counts) arr.fill(0);
113
114     for(const auto &stu : students) {
115         for(int c = 0; c < courseCount; c++) {
116             int score = stu.scores[c];
117             if(score == 0) counts[c][0]++;
118             else if(score > 0 && score < 60) counts[c][1]++;
119             else if(score >= 60 && score < 75) counts[c][2]++;
120             else if(score >= 75 && score < 90) counts[c][3]++;
121             else if(score >= 90 && score <= 100) counts[c][4]++;
122         }
123     }
124
125     int total = static_cast<int>(students.size());
126     cout << "\n各课程成绩段人数及占比：" << endl;
127     cout << left << setw(12) << "课程";
128     cout << setw(10) << "0" << setw(12) << "(0,60)"
129         << setw(12) << "[60,75)" << setw(12) << "[75,90)" << setw(12)
130         << "[90,100]" << "\n";
131
132     for(int c = 0; c < courseCount; c++) {
133         cout << left << setw(12) << ("Course" + to_string(c + 1));
134         for(int s = 0; s < segmentCount; s++) {
135             int cnt = counts[c][s];
136             double pct = total > 0 ?
137                 (static_cast<double>(cnt) * 100.0 / total)
138                 : 0.0;
139             cout << setw((s == 0) ? 10 : 12) <<
140                 (to_string(cnt) + "(" + to_string
```

```
141         ((int)round(pct)) + "%)");
142     }
143     cout << "\n";
144 }
145 }
146
147 int main() {
148     system("chcp 65001 > nul");
149     int n;
150     cout << "请输入学生数量：";
151     cin >> n;
152     if(n <= 0) {
153         cerr << "学生数量必须大于0。" << endl;
154         return 1;
155     }
156     if(n > 100) {
157         cerr << "最大支持100名学生。" << endl;
158         return 1;
159     }
160
161     vector<StudentGrade> students;
162     students.reserve(n);
163     for(int i = 0; i < n; i++) {
164         StudentGrade stu;
165         cout << "\n请输入第" << i + 1 << "名学生的学号：";
166         cin >> stu.id;
167         cout << "请输入第" << i + 1 << "名学生的姓名：";
168         cin >> stu.name;
169         cout << "请输入第" << i + 1 << "名学生的籍贯：";
170         cin >> stu.hometown;
171         cout << "请输入第" << i + 1 << "名学生的专业：";
172         cin >> stu.major;
173         int sum = 0;
174         for(int j = 0; j < 5; j++) {
175             cout << "请输入第" << i + 1 << "名学生第"
176                 << j + 1 << "门课程成绩：";
177             cin >> stu.scores[j];
178             sum += stu.scores[j];
179         }
180         stu.average = static_cast<double>(sum) / 5.0;
181         students.push_back(stu);
```

```
182     }
183
184     const char *fileName = "StudentGrade.dat";
185     if(!saveStudents(fileName, students)) {
186         cerr << "无法创建二进制文件 " << fileName << endl;
187         return 1;
188     }
189     cout << "已将 " << n << " 名学生的全部数据保存到 "
190          << fileName << endl;
191
192     vector<StudentGrade> loaded;
193     if(!loadStudents(fileName, loaded)) {
194         cerr << "无法读取二进制文件 " << fileName << endl;
195         return 1;
196     }
197
198     reportGroupStatistics(loaded, true); // 按籍贯
199     reportGroupStatistics(loaded, false); // 按专业
200     reportGradeSegments(loaded);
201
202     return 0;
203 }
```

```
请输入第1名学生的学号: 87
请输入第1名学生的姓名: asf
请输入第1名学生的籍贯: 黑龙江
请输入第1名学生的专业: 软件
请输入第1名学生第1门成绩: 65
请输入第1名学生第2门成绩: 87
请输入第1名学生第3门成绩: 56
请输入第1名学生第4门成绩: 46
请输入第1名学生第5门成绩: 86

请输入第2名学生的学号: 90
请输入第2名学生的姓名: si
请输入第2名学生的籍贯: 包头
请输入第2名学生的专业: 计算机
请输入第2名学生第1门成绩: 56
请输入第2名学生第2门成绩: 35
请输入第2名学生第3门成绩: 45
请输入第2名学生第4门成绩: 95
请输入第2名学生第5门成绩: 15

请输入第3名学生的学号: 36
请输入第3名学生的姓名: uhs
请输入第3名学生的籍贯: 江西
请输入第3名学生的专业: 软件
请输入第3名学生第1门成绩: 34
请输入第3名学生第2门成绩: 56
请输入第3名学生第3门成绩: 67
请输入第3名学生第4门成绩: 35
请输入第3名学生第5门成绩: 48

请输入第4名学生的学号: 94
请输入第4名学生的姓名: jis
请输入第4名学生的籍贯: 南昌
请输入第4名学生的专业: 34
请输入第4名学生第1门成绩: 45
请输入第4名学生第2门成绩: 67
请输入第4名学生第3门成绩: 35
请输入第4名学生第4门成绩: 76
请输入第4名学生第5门成绩: 43
已将 4 名学生的全部数据保存到 StudentGrade.dat
```

图 6: 实验六运行结果

按籍贯分组的课程成绩统计：

籍贯：包头

课程	最小	最大	均值	中位数
Course1	56	56	56.00	56.00
Course2	35	35	35.00	35.00
Course3	45	45	45.00	45.00
Course4	95	95	95.00	95.00
Course5	15	15	15.00	15.00

籍贯：南昌

课程	最小	最大	均值	中位数
Course1	45	45	45.00	45.00
Course2	67	67	67.00	67.00
Course3	35	35	35.00	35.00
Course4	76	76	76.00	76.00
Course5	43	43	43.00	43.00

籍贯：江西

课程	最小	最大	均值	中位数
Course1	34	34	34.00	34.00
Course2	56	56	56.00	56.00
Course3	67	67	67.00	67.00
Course4	35	35	35.00	35.00
Course5	48	48	48.00	48.00

籍贯：黑龙江

课程	最小	最大	均值	中位数
Course1	65	65	65.00	65.00
Course2	87	87	87.00	87.00
Course3	56	56	56.00	56.00
Course4	46	46	46.00	46.00
Course5	86	86	86.00	86.00

图 7: 实验六运行结果

按专业分组的课程成绩统计:

专业: 34

课程	最小	最大	均值	中位数	
Course1	45	45	45.00	45.00	
Course2	67	67	67.00	67.00	
Course3	35	35	35.00	35.00	
Course4	76	76	76.00	76.00	
Course5	43	43	43.00	43.00	

专业: 计算机

课程	最小	最大	均值	中位数	
Course1	56	56	56.00	56.00	
Course2	35	35	35.00	35.00	
Course3	45	45	45.00	45.00	
Course4	95	95	95.00	95.00	
Course5	15	15	15.00	15.00	

专业: 软件

课程	最小	最大	均值	中位数	
Course1	34	65	49.50	49.50	
Course2	56	87	71.50	71.50	
Course3	56	67	61.50	61.50	
Course4	35	46	40.50	40.50	
Course5	48	86	67.00	67.00	

各课程成绩段人数及占比:

课程	0	(0,60)	[60,75)	[75,90)	[90,100]
Course1	0(0%)	3(75%)	1(25%)	0(0%)	0(0%)
Course2	0(0%)	2(50%)	1(25%)	1(25%)	0(0%)
Course3	0(0%)	3(75%)	1(25%)	0(0%)	0(0%)
Course4	0(0%)	2(50%)	0(0%)	1(25%)	1(25%)
Course5	0(0%)	3(75%)	0(0%)	1(25%)	0(0%)

图 8: 实验六运行结果